

SW Team - Coding Guidelines

Table of Content

- [Repositories](#)
 - [General](#)
 - [Commit messages](#)
- [Generic Coding Guidelines](#)
- [Code Comments](#)
- [C coding \(cc\)](#)
 - [type casts \(cast\)](#)
 - [const keyword \(const\)](#)
 - [Include guard style \(include\)](#)
 - [control flow \(flow\)](#)
 - [RAII in plain C \(raii\)](#)
 - [Code formatter \(formattool\)](#)
 - [Format \(format\)](#)
 - [Typedef'd types \(typedef\)](#)
 - [Naming \(naming\)](#)
 - [Multiple instances \(multinst\)](#)
 - [C features \(feature\)](#)
- [Attachments](#)



Attention!

This is the current version of the coding guidelines locked and updated on 17 Mar 2020. Changes can be proposed at all times but must be approved prior by everyone and trigger another round of code review for all work in progress components.

For proposals, please enter your request [in the table on the subpage](#) with status pending and your name. The requests will be discussed during one of the next SCRUM Meetings. Please wait with applying your new rule until it is approved. Once approved the change will be added to this Wiki page.

Repositories

General

- Every repository should contain a readme file describing it.
- Until we really need one: do not use a versioning scheme for the component contained in a repository; git (submodule = using commits) is sufficient for the time being.
- A repository (containing a component) optionally provides a way of building a corresponding library. In any case other repositories using it are free to cherry pick files.
- The code for tests (unit tests (usually running on Linux or integration tests (usually running on seL4)) should be contained in the same repository as the component they are intended to test. The actual project running these tests (= creating a working test environment for them) should be contained in a separate repository.
- A repository should contain a top level include folder containing the public interface of the component. Private include files can be placed in the "src" (for example) folder.
- A repository should use a top level folder containing all external dependencies. The folder "project" contains only components being built with the seL4 build system.

Commit messages

- commit format should be "<TAG> <JIRA Ticket ID> <message>"
 - eg. "[DEV] SEOS-42 added foobincarnation".
 - The JIRA ticket ID is optional.
 - TAG should be short (avoid more than 3 chars) to describe the scope:
 - **DEV**: for generic development commits. Please consider using these tags to be more specific:
 - **FIX**: for commit that fix a bug, ideally a JIRA ticket is mentioned also.
 - **NEW**: for commit that prepare the ground for something new.
 - **REM**: for commits that remove obsolete things.
 - **REF**: for refactoring, where the functionality does not change.
 - **FOR**: for code formatting, where the functionality does no change.
 - **DOC**: for documentation updates, fixing typos
 - **CI**: for any changes related to the CI system or general configuration
 - **TA**: for any changes related to the run of the tests
 - **BUILD**: for any changes relate to the build script and the relates make system
 - ... list to be extended with proposals. Note that we have rejected the tag BUG and use FIX instead.
- commit message shall be precise and describe what happen semantically.
 - They should avoid stating obvious things like "changed foo.c"
 - better say what changing there actually, e.g. "replace foobincarnation by barification"
- for squashed commits, rewrite the commit messages
 - do not just aggregate all messages from all the commit that have been squashed nobody cares about the commit IDs of all the squashed commit in the end.

Generic Coding Guidelines

- write readable code
- don't write code that is never used

- code shall have zero warnings, even when hacking something in quickly. Each warning must have comment explaining it. If pragmas are used to avoid warning, there must be a comment explaining things
- 80 char line length includes indentation
- avoid busy loop polling things
 - polling loops need comment explaining why they are acceptable as busy loop
 - a while(1) loop is no way to end a thread, use the proper OS mechanism here or block on some even that never happens, so the OS can suspend the thread
 - if a yield-loop is used to wait for a while, this needs a proper explanation why this is needed - any why there is no other way to solve this with explicit synchronization

Code Comments

- when in doubt, write a comment
- when looking at your code and something is not clear, place a comment there
- write comment about the current state of the code
 - avoid comment that say "// changed a from x to y". We've see this happening in code review cycles in response to a remark. Comment like this are commit messages., but not code comments. Clearly state facts about the state of the code like "The common default x does not work because ... and current test show that y is sufficient"
- explain choices for constants that are not trivial, e.g. values for timeouts of buffer sizes.
 - Clarify if there are dependencies here.
 - It's perfectly valid to say in a comment that it's an arbitrary values. Or it's unknown why this values was chosen. Or that timeouts were chosen because of experimental exploration
- write error message that provide all details for the target audience
 - most log are for technical people, not for end users
 - print the value that is not as expected (unless security reasons forbid this)
 - write compact error message stating the issue, so the log contains something like "foo() failed for handle 27, code 5"
 - don't write verbose messages like "There was a failure in the function Foo. The handle provided was 27. The error code was 5."
- All module's public interfaces shall be commented with the doxygen style comments.

C coding (cc)

- https://en.wikipedia.org/wiki/The_Power_of_10:_Rules_for_Developing_Safety-Critical_Code

type casts (cast)

cc.cast.1

- use explicit casts when ever needed.
- fixing broken interfaces and signatures before casting data and thus potentially breaking things.
- create tickets if broken interface can't be fixed, add a comment in the code that casting is required because of the broken interface
- function taking a data buffer should use "void*" to accept anything, don't use "uint8_t*" or "char*" there.

const keyword (const)

cc.const.1

- use const whenever possible, especially when defining and passing pointers to memory that shall not be modified.
- See also [Misc Tool Tips#Cconstpointers](#)

Include guard style (include)

cc.include.1

"pragma once" is preferred over define, rationale is, that practically every compiler supports this today and it exists exactly to solve the problem.

- discussion
 - <https://stackoverflow.com/questions/1143936/pragma-once-vs-include-guards>
 - https://en.wikipedia.org/wiki/Pragma_once
- guideline:

Preferred	Accepted for legacy code
-----------	--------------------------

⊖ not in ANSI C standard + lightweight + practically supported by every compiler + method exists to provide solution for the include guard problem <pre>#pragma once ...</pre>	+ complies with ANSI C standard ? people are used to this ⊖ overhead ⊖ may fail if two headers use the same define ⊖ creates defines that are usually never needed. Potential pitfall if other file start to use them instead of checking specific feature-defines <pre>#ifndef MY_HEADERFILE #define MY_HEADERFILE ... #endif // MY_HEADERFILE</pre>
--	---

control flow (flow)

cc.flow.1

return statements in if-blocks

don't create an else-block if the if-block returns. There is no control flow that merges after the if-else-construct and the code there does not have to worry about earlier problems at all

Good	Bad
<pre>if (isError) { return -1; } do_something(); do_something_else();</pre>	<pre>if (isError) { return -1; } else { do_something(); } do_something_else();</pre>

cc.flow.2

- Consider returning detailed error codes instead of just true/false. Log a message when returning an error
- Function may hide error details for security reasons, however, thus must be states explicitly in the code

cc.flow.3

Avoid using #ifdef within a function, better have a helper function that hides this from the function.

cc.flow.4

Consider handling the error case first and return. So there is no intention for the good-case control flow that continues through the function.

RAII in plain C (raii)

cc.raii.1

since GCC 4.0, there is the attribute "cleanup".

Docs says: "The `cleanup` attribute runs a function when the variable goes out of scope. This attribute can only be applied to auto function scope variables; it may not be applied to parameters or variables with static storage duration. The function must take one parameter, a pointer to a type compatible with the variable. The return value of the function (if any) is ignored. If `-fexceptions` is enabled, then `cleanup_function` is run during the stack unwinding that happens during the processing of the exception. Note that the `cleanup` attribute does not allow the exception to be caught, only to perform an action. It is undefined what happens if `cleanup_function` does not return normally."

```
#include <stdio.h>

void my_int_finalizer(int * pVal)
{
    printf("variable (%d) goes out of scope\n", *pVal);
}

int main(void)
{
    printf("before scope\n");

    {
        int watched __attribute__((cleanup(my_int_finalizer))) = 42;
    }

    printf("after scope\n");
}
```

This prints:

```
before scope
variable (42) goes out of scope
after scope
```

Code formatter (formattool)

cc.formattool.1

we have astyle for this, it runs as parts of the build process. Our script was derived from https://github.com/mellowcandle/astyle_precommit_hook

```
ASTYLE_SETTINGS=" \
--suffix=none \
--style=allman \
--indent=spaces=4 \
--indent-classes \
--indent-namespaces \
--pad-oper \
--pad-header \
--pad-comma \
--add-brackets \
--align-pointer=type \
--align-reference=name \
--min-conditional-indent=0 \
--lineend=linux \
--max-code-length=80 \
--max-continuation-indent=60"
```

Format (format)

cc.format.1

Use white space, not tabs

- Indention should be 4 white spaces for code.
- no whitespace at the end of a line. Configure your editor to remove them automatically
- the only exception are makefiles, as tabs are part of the syntax there. But we use cmake and ninja anyway.

cc.format.2

Line width

- should be about 80 characters
- please, try not to make line longer. Sometimes it is hard to do, especially when dealing with long function or symbol names.
- it's all about code readability, and especially when reviewing code snippets and patches in an environment that is not the usual screen or editor.
- 80 may be an obscure value from ancient computer history. But it's not this bad, given hundreds of years of typography have concluded that the average human prefers just 50-60 characters per line for text

cc.format.3

- Formatting function signatures (declaration and definition):
 - parameter on next line
 - indented by 4 space

cc.format.4

- use explicit curly brackets for every code block, even if it is a one-line where they are not strictly necessary

Good	Bad
<pre>if (is_foo) { bar(); }</pre>	<pre>if (is_foo) bar();</pre>

Typedef'd types (typedef)

cc.typedef.1

should end with a "_t" or some other appropriate suffix, especially structs. If the struct itself has a name, it should start with an underscore.

```
typedef struct _foo {
    int something;
} foo_t, *foo_ptr;
```

Naming (naming)

cc.naming.1

Names of variables and function

- should start with a lower case char. They may use camel-case or underscores, whatever fits better. Naming must just be consistent within a module

cc.naming.2

Global function naming

C functions defined in a module within an application must be named according to the following schema: Module_function

Where Module is the name of the module/class (CamelCase, first character upper) and function is the name of the function/method (camelCase, first character lower).

In C language only a global scope exists for function names, that means that the more functions are added to an application, the more this space gets polluted and the more the chances for a name collision. Also this global namespace allows a certain degree of ambiguity (e.g. draw is the method for drawing a shape on the screen or the method to extract a card from a deck?). The programmer may find herself puzzled over which function has to be used to accomplish the task she is trying to implement. Other, more recent, languages have faced the problem and solved by providing some means to split the global name space:

- * Java provides packages,
- * C++ provides namespaces

Multiple instances (multinst)

cc.multinst.1

In C it is possible to achieve this in a clear and OO fashion by applying the following discipline. A module name is like a class name and the main type exported by that module is a structure with the same name of the module where the per-instance context is stored. Every public function has a prototype with the pointer to the instance's context as a first argument. The module shall provide a constructor and a destructor function.

- Code reuse
- Testability
- Copy&Paste anti-pattern avoidance

In ModuleName.h

```
typedef struct
{
    ...
}
ModuleName;

bool ModuleName_ctor(ModuleName* self, ...);
void ModuleName_dtor(ModuleName* self);
```

In ModuleName.c

```
#include "ModuleName.h"

...
```

C features (feature)

cc.feature.1

- avoid using "inline" blindly
 - nowadays it often is just a compiler hint that it may use - or ignore.
 - the selected optimization level is crucial here, don't expect any inlining to happen at "-O0".
 - do not "optimize C code", but give the compiler all knowledge and constraints (attributes, intrinsics ...) to do this job for you.
- if function inlining is really needed
 - there are ways to enforce inlining, read the compiler doc carefully. See <https://gcc.gnu.org/onlinedocs/gcc/Inline.html>
 - clarify and document in detail, what issues are addressed by inlining
 - verify that the expected inlining really happens - and that the inlined code really solves the problem
- the "inline" keyword has a semantic impact beyond optimization
 - this function will be defined in multiple translation units, don't worry about it. The linker needs to make sure all translation units use a single instance of the variable/function.
 - especially, without the inline keyword, one may have to use `__attribute__((unused))` to avoid a warning about unused functions
 - similar semantics exists for other keywords
 - static: variable/function name cannot be used in other translation units. Linker needs to make sure it doesn't accidentally use a statically defined variable/function from another translation unit.
 - extern: use variable/function name in this translation unit but don't complain if it isn't defined. The linker will sort it out and make sure all the code that tried to use some extern symbol has its address.

cc.feature.2

- Functions without parameters must be declared as "foo(void)" and not "foo()".
- Rational is that the C standard says that "foo()" means the function can take any parameters. "foo(void)" explicitly says that the function does not take any parameter.

cc.feature.3

- Do not use "#if 0" in the code. Either we have a define that controls this or a comment or there is nothing at all. Comment the code out instead
- If you use "#if 0" for development branches, it's good practice to add a comment there why this code block is disabled.

Attachments

File	Modified
File pre-commit-hook.sh	Oct 30, 2018 by HG123912